Mersenne Tabanlarında Pascal Satırları ve En Büyük Eldesiz Satır

On örnek ve adresler üzerinden işlem modeli

James Augustus

27 Ekim 2026

CC BY 4.0. Telif hakkı © 2026 James Augustus.

İçindekiler

Öz 3

1. Taban ve hane kavramı 4

2. Pascal satırları ve en büyük katsayı 5

3. Pascal katsayılarının kuvvetlerle ilişkisi 6

4. İlk çalışmanın seçtiği satır 7

5. N(p) sorusu ve p = 38 örneği 8

6. Eşikler ve iki çalışmanın farkı 9

7. Yaklaşık büyüklük ve kanıt sınırı 10

8. On örnekle N(p) ne işe yarar? 11

9. Adreslerden hücrelere: PCCPU ile işlem modeli 15

10. Yeniden üretim ve sonuç 19

Öz

Bu metin, Pascal satırlarının bir sayı tabanında hane dizisi olarak kullanılmasını ve bu kullanımın sınırını inceler. Birinci problem, seçilen bir satırın eldesiz olup olmadığının gösterilmesidir. İkinci problem, aynı tabanda eldesiz kalabilen son satırın bulunmasıdır. İlk problemde n = p + 2 satırı seçilir. İkinci problemde taban sabit tutulur ve satır numarası değiştirilir. Bu son satırın numarası N(p) ile gösterilir.

İki problem aynı binom özdeşliğine dayanır. Bununla birlikte aynı soruya cevap vermez. p ≥ 7 için p + 2 satırının geçerli olduğu bilinir. Bu satırın her zaman son geçerli satır olduğu söylenemez. p = 38 örneğinde p + 2 = 40 satırı geçerlidir, fakat son geçerli satır 41’dir.

Açıklama, önce taban ve hane kavramlarını kurar. Ardından Pascal üçgeni, elde taşıma koşulu, elle hesaplanabilen örnekler, taban seçimi ve N(p) eşikleri ele alınır. Kanıtlanmış sonuçlar, sonlu hesap kontrolleri ve açık kalan önerme birbirinden ayrılır. Ayrıntılı kanıtlar, deponun özgün matematik notu ve N(p) ekinde bulunur [1, 2].

Anahtar kelimeler: Pascal üçgeni, Mersenne tabanı, binom katsayısı, eldesiz gösterim, merkez katsayısı.

Simge	Anlamı
p	Tabanı belirleyen tamsayı parametre
$b = 2^p - 1$	Kullanılan sayı tabanı
n	İncelenen Pascal satırının numarası
$C(n,k)$	n satırının k konumundaki binom katsayısı
$c_n = C(n,\lfloor n/2 \rfloor)$	n satırındaki en büyük katsayı
$N(p)$	b tabanındaki son eldesiz satırın numarası
$D(p) = N(p) - p$	Son satır ile p arasındaki fark

1. Taban ve hane kavramı

1.1. Bir sayının hanelerle gösterilmesi

Onluk tabanda 347 sayısı, $3 \times 100 + 4 \times 10 + 7$ biçiminde açılır. Hanelerin değeri kadar konumu da önemlidir. Aynı kural b tabanında da geçerlidir. Soldan sağa a_2, a_1, a_0 haneleri, $a_2b^2 + a_1b + a_0$ sayısını gösterir.

Geçerli bir hane 0 ile $b - 1$ arasındaki bir tamsayıdır. Hane değeri b'ye ulaştığında bir üst konuma elde aktarılır. Onluk tabanda 10'un tek hane olmaması bu kuralın özel bir örneğidir.

Büyük tabanlar için haneleri onluk yazımlarıyla ve aralarında ayırıcı bırakarak göstermek uygundur. 127 tabanında 126, tek bir hane değeridir. Onluk yazımında üç rakam bulunması, 127 tabanında üç hane olduğu anlamına gelmez.

Taban ve haneler	Onluk karşılık
127 tabanında [1, 2]	$1 \times 127 + 2 = 129$
127 tabanında [1, 126]	$1 \times 127 + 126 = 253$
127 tabanında [1, 127]	İkinci değer geçerli hane değildir. Elde sonrası [2, 0] olur.

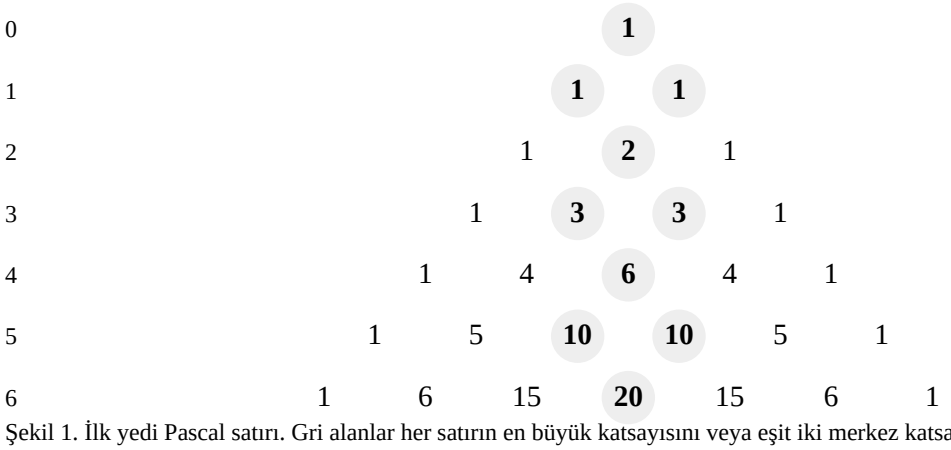
1.2. Eldesiz olmanın anlamı

Bir katsayı dizisinin eldesiz gösterim oluşturması, bütün katsayıların tabandan küçük olması demektir. Eldeli ve eldesiz açılımlar aynı sayısal değeri verebilir. Ancak elde oluştuğunda başlangıçtaki katsayı dizisi, sayının gerçek hane dizisi olarak korunmaz.

Bu çalışmada araştırılan özellik, sayısal eşitliğin tek başına doğruluğu değildir. Pascal katsayılarının değiştirilmeden gerçek hane olarak kullanılabilmesidir. Bu nedenle taban sınırı kesin bir eşitsizlikle yazılır: her k için $C(n,k) < b$.

2. Pascal satırları ve en büyük katsayı

Pascal üçgeninin başlangıç satırı n = 0 olarak alınır. Her satırın iki ucu 1’dir. İçteki her sayı, üst satırdaki komşu iki sayının toplamıdır. n numaralı satırda n + 1 katsayı bulunur.



Şekil 1. İlk yedi Pascal satırı. Gri alanlar her satırın en büyük katsayısını veya eşit iki merkez katsayısını gösterir.

Satır numarası	Katsayılar
0	1
1	1, 1
2	1, 2, 1
3	1, 3, 3, 1
4	1, 4, 6, 4, 1
5	1, 5, 10, 10, 5, 1

Satırlar simetriktir. En büyük katsayı ortadadır. Satır numarası çift olduğunda tek merkez, tek olduğunda eşit iki merkez katsayısı bulunur. Örneğin 4. satırın merkezi 6, 5. satırın iki merkez katsayısı 10’dur.

Bu özellik, eldesizlik kontrolünü sadeleştirir. Satırın en büyük katsayısı tabandan küçükse diğer katsayıların tamamı da küçüktür. Dolayısıyla bütün satırı karşılaştırmak yerine $c_n = C(n,\lfloor n/2 \rfloor)$ değerinin tabana göre konumunu incelemek yeterlidir.

$n \geq 1$ için merkez katsayıları kesin olarak artar. Bu nedenle bir tabanda sınır aşıldıktan sonra daha sonraki satırlarda yeniden eldesiz duruma dönülmez. Son eldesiz satır kavramı bu monotonluk nedeniyle anlamlıdır.

3. Pascal katsayılarının kuvvetlerle ilişkisi

3.1. Binom özdeşliği

Binom teoremi, $(b + 1)^n$ açılımının katsayılarını Pascal üçgeninden verir. Örneğin n = 4 için sonuç şöyledir:

$(b + 1)^4 = b^4 + 4b^3 + 6b^2 + 4b + 1$.

Bu açılımın katsayıları [1, 4, 6, 4, 1] dizisidir. Bütün katsayılar b'den küçük olduğunda bu dizi, sonucun b tabanındaki hanelerini doğrudan verir. Aksi durumda özdeşlik yine doğrudur, ancak haneler elde taşıma nedeniyle değişir.

Onluk tabanda $11^2 = 121$, $11^3 = 1331$ ve $11^4 = 14641$ sonuçları bu ilişkiyi gösterir. Buna karşılık $11^5 = 161051$ olur. Beşinci Pascal satırındaki iki adet 10, onluk tabanda hane olamadığı için [1, 5, 10, 10, 5, 1] dizisi doğrudan korunmaz.

3.2. Mersenne tabanının seçilmesi

Taban $b = 2^p - 1$ seçildiğinde $b + 1 = 2^p$ olur. Binom özdeşliği bu durumda $(b + 1)^n = 2^{pn}$ sonucunu verir. Eldesizlik koşulu sağlanıyorsa 2^{pn} sayısının b tabanındaki haneleri n. Pascal satırıdır.

Burada $p \geq 2$ tamsayı bir parametredir. p'nin asal olması veya $2^p - 1$ sayısının asal olması genel eldesizlik sorusunun koşulu değildir. Örneğin ileride kullanılan p = 38 bileşiktir. Bu gösterim bir asallık testi oluşturmaz.

İki ayrı ifade birlikte korunmalıdır. Sayısal özdeşlik her geçerli b ve n için doğrudur. Katsayıların hane olarak aynen kalması ise ayrıca $c_n < b$ koşulunu gerektirir.

3.3. Küçük bir örnekte eldeyi görmek

p = 3 seçildiğinde taban 7 olur. Dördüncü Pascal satırı [1, 4, 6, 4, 1] bu tabana sığar. Sayının değerini elle hesaplayabiliriz:

$1 \times 7^4 + 4 \times 7^3 + 6 \times 7^2 + 4 \times 7 + 1 = 2401 + 1372 + 294 + 28 + 1 = 4096$.

Bu sonuç $8^4 = 2^{12}$ sayısıdır. Beşinci satır ise [1, 5, 10, 10, 5, 1] olur. 7 tabanında 10 tek bir hane olamaz. Sağdan sola elde uygulanınca haneler değişir:

İşlem	Hanede kalan	Sola taşınan
Sağdaki $10 = 1 \times 7 + 3$	3	1
Soldaki $10 + 1 = 11 = 1 \times 7 + 4$	4	1
Yanındaki $5 + 1 = 6$	6	0

Sonuç [1, 6, 4, 3, 5, 1] olur. Sayının değeri hâlâ $8^5 = 32768$'dir. Kaybolan şey sayısal eşitlik değil, Pascal satırının aynen hane dizisi olarak kalmasıdır. Bu yüzden N(3) = 4 olur. Tanım p = 3 için geçerlidir, ancak p + 2 güvencesi yalnız $p \geq 7$ içindir.

4. İlk çalışmanın seçtiği satır

Özgün Pascal Mersenne Rows çalışmasında $n = p + 2$ satırı ele alınır [1]. Kanıtlanan sonuç, her $p \geq 7$ tamsayısı için bu satırın $2^p - 1$ tabanında eldesiz olmasıdır. Böylece belirli bir satır için genel bir geçerlilik güvencesi elde edilir.

$p = 7$ seçildiğinde $b = 127$ ve $n = 9$ olur. Dokuzuncu satır şöyledir:

```
1 9 36 84 126 126 84 36 9 1
```

En büyük katsayı 126’dır. Bu değer 127’den küçük olduğundan on katsayının tamamı geçerli hanedir. Hanelerin konum değerleriyle toplanması $2^{7\times 9} = 2^{63}$ sonucunu verir.

Satır	En büyük katsayı	127 tabanında durum
8	70	Eldesiz
9	126	Eldesiz
10	252	Elde gerekir

Bu örnekte ilk çalışmanın seçtiği 9. satır aynı zamanda son eldesiz satırdır. Dolayısıyla $N(7) = 9$ olur. Ancak bu tek örnek, $p + 2$ satırının her p için son satır olduğunu göstermez.

Özgün not bu ayrımı zaten içerir. $7 \leq p \leq 37$ aralığında $p + 2$ son eldesiz satırdır. $p = 38$ için $p + 3$ satırının da geçerli olduğu belirtilir. Yeni $N(p)$ incelemesi, önceki notta hiç bulunmayan bir ayrımı ilk kez ileri sürmekten çok, bu sınır sorusunu sistematik olarak genişletir.

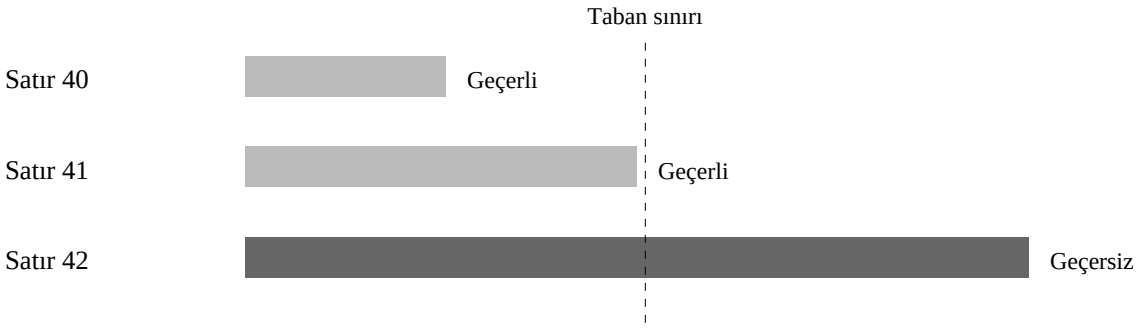
5. N(p) sorusu ve p = 38 örneği

5.1. Tabanı sabit tutarak satırı değiştirmek

N(p), $C(n,\lfloor n/2 \rfloor) < 2^p - 1$ koşulunu sağlayan en büyük n tamsayısıdır. N(p) hesaplanırken p ve buna bağlı taban sabit tutulur. Değiştirilen büyüklük yalnız satır numarasıdır.

p = 38 için taban 274877906943’tür. İlk çalışmanın seçtiği satır 40’tır. Bu satır ile sonraki iki satırın merkezleri aşağıda karşılaştırılır.

Satır n	Merkez katsayısı c_n	Sonuç
40	137846528820	Tabandan küçük
41	269128937220	Tabandan küçük
42	538257874440	Tabandan büyük



Şekil 2. p = 38 için merkez katsayılarının taban sınırına göre büyüklüğü. 41. satır geçerlidir, 42. satır sınırı aşar.

Sonuç N(38) = 41’dir. İlk çalışma 40. satırın geçerli olduğunu söyler. Sınır incelemesi, aynı tabanda bir satır daha ilerlenebildiğini belirler. Bu iki sonuç birbiriyle çelişmez.

5.2. Aynı taban, farklı sayısal değer

40. satırın haneleri $2^{38\times40} = 2^{1520}$ sayısını gösterir. 41. satırın haneleri ise $2^{38\times41} = 2^{1558}$ sayısını gösterir. Taban aynıdır, temsil edilen sayılar farklıdır.

N(p), o tabanda yazılabilecek en büyük sayıyı ifade etmez. Her tabanda daha fazla hane kullanılarak daha büyük sayılar yazılabilir. N(p) yalnızca Pascal katsayılarının elde taşımadan korunabildiği satır ailesinin sınırıdır.

6. Eşikler ve iki çalışmanın farkı

6.1. Son satırın basamaklı değişimi

D(p) = N(p) − p farkı belirli aralıklarda sabit kalır. İlk aralıklar şöyledir [2]:

p aralığı	Son eldesiz satır	p + 2’ye göre ek satır
7-37	N(p) = p + 2	0
38-158	N(p) = p + 3	1
159-645	N(p) = p + 4	2
646-2600	N(p) = p + 5	3
2601-10421	N(p) = p + 6	4
10422-41712	N(p) = p + 7	5

Örneğin N(100) = 103 ve N(1000) = 1005 olur. İlk seçilen satırlar sırasıyla 102 ve 1002’dir. D(p) farkı ile ilk çalışmaya göre kazanılan ek satır sayısı aynı değildir. İkincisi N(p) − (p + 2), yani D(p) − 2’dir.

N(37) = 39’dan N(38) = 41’e geçiş, sabit bir tabanda 40. satırın atlanması anlamına gelmez. p değiştiğinde taban da değişir. Bu iki N değeri farklı tabanlara aittir.

6.2. Kesin eşik tanımı

$k \geq 2$ için m_k, $2^k C(2m,m) < 4^m$ eşitsizliğini sağlayan en küçük m tamsayısı olarak tanımlanır. Eşik $p_k = 2m_k - k - 1$ ile elde edilir. $p \geq 7$ için D(p), $p_k \leq p$ olan en büyük k değeridir. Ayrıntılı türetim N(p) notundadır [2].

Burada tanımın kesin olması ile kısa bir kapalı formun kanıtlanmış olması farklıdır. $m_k = \lceil 4^k/\pi - 1/4 \rceil$ ifadesi hesapla sınanmış bir adaydır. k = 1,…,12 kontrolleri bu adayı destekler. Bütün k değerleri için geçerli olduğu bu pakette kanıtlanmış kabul edilmez.

Başlık	Özgün çalışma	N(p) eki
Soru	p + 2 satırı geçerli mi?	Son geçerli satır hangisi?
Satır seçimi	n = p + 2 önceden seçilir.	n, sınır bulunana kadar değişir.
Ortak temel	Binom özdeşliği ve merkez katsayısı	Aynı özdeşlik ve katsayı
Sonuç türü	Belirli satır için genel güvence	Son satır, eşikler ve büyüme sınırı

7. Yaklaşık büyüklük ve kanıt sınırı

Merkez binom katsayılarının büyüklüğü, son satır için aşağıdaki ifadeye götürür [2]:

$N(p) = p + (1/2)\log_2(\pi p/2) + E(p)$.

$p \geq 7$ için hata terimi $-1 < E(p) \leq 2 - (1/2)\log_2(7\pi/2)$ sınırını sağlar. Üst sınır yaklaşık 0,270574474235’tir. 0,270575 yazımı, sınırı dışarı doğru yuvarlayan bir gösterimdir. Tam üst sınır ile yuvarlatılmış ondalık sayı aynı şey değildir.

Bu bağıntı, p üzerindeki artışın yavaş büyüdüğünü gösterir. $p = 1000$ için hata terimi dışındaki ifade yaklaşık 1005,309 olur. Gerçek sonuç 1005’tir. Yaklaşık değerin tek bir örnekte yakın tamsayıya yuvarlanması, her durumda aynı yuvarlama kuralının çalıştığını kanıtlamaz.

Pakette $p = 7,\ldots,20000$ ve eşikler için $k = 1,\ldots,12$ aralıkları doğrudan hesapla denetlenmiştir. Bu, sonlu bir kontroldür. Kanıtlanmış eşik karakterizasyonu ile belirli eşiklerin kesin hesaplanması daha geniş aralıklara sonuç taşımaya izin verebilir. Ancak bu tür bir çıkarım, o geniş aralıktaki her p değerinin tek tek tarandığı anlamına gelmez.

Özgün çalışmadaki bazı Catalan ve çarpan ilişkileri, p’nin tek olması ve $n = p + 2$ seçimine bağlıdır. Satırın N(p) ile değiştirilmesi bu koşulları otomatik olarak korumaz. Bu nedenle özgün özel ilişkiler yeni satıra ayrıca kanıtlanmadan aktarılmamalıdır.

8. On örnekle N(p) ne işe yarar?

N(p), Pascal katsayılarını bir sayının haneleri olarak kullanırken geçerlilik sınırını verir. Taban seçilmişse hangi satıra kadar gidilebileceğini söyler. Ters yönde, belirli bir satırı elde taşımadan yazmak için yeterli tabanı seçmeye yardım eder. Aşağıdaki örnekler bu kullanımları gösterir. Bunlar yeni bir hız veya sıkıştırma sonucu değildir.

Örnek 1. Elle kontrol edilebilen bir sayı

p = 3 için taban 7'dir. Dördüncü Pascal satırı [1, 4, 6, 4, 1] olur. En büyük katsayı 6 < 7 olduğu için hepsi geçerli hanedir. Gerçekten

$1 \times 7^4 + 4 \times 7^3 + 6 \times 7^2 + 4 \times 7 + 1 = 4096 = 8^4 = 2^{12}$.

Beşinci satır [1, 5, 10, 10, 5, 1] olur. 10, 7 tabanında hane değildir. Aynı katsayı açılımının değeri hâlâ $8^5 = 32768$'dir, fakat gerçek haneler [1, 6, 4, 3, 5, 1] olur. Sağdaki 10, 3 kalanı ve 1 elde verir. Soldaki 10 bu eldeyle 11 olur, 4 kalanı ve 1 elde verir. Yanındaki 5 de 6 olur. Bu yüzden N(3) = 4. Buradaki p = 3 örneği tanımı açıklar, $p \geq 7$ isteyen Teorem A'nın kapsamına girmez.

Örnek 2. Sınırdaki eşitlik de başarısızdır

p = 2 için taban 3'tür. İkinci satır [1, 2, 1] sığar. Üçüncü satır [1, 3, 3, 1] sığmaz, çünkü geçerli haneler yalnız 0, 1 ve 2'dir. Dolayısıyla N(2) = 2. Kontrol $c(n) \leq b$ değil, $c(n) < b$ olmalıdır. Bir program bu eşitlik durumunu kabul ederse yanlış hane dizisi üretir.

Örnek 3. 127 tabanında bütün satırı görmek

p = 7 için taban 127'dir. 9. satır [1, 9, 36, 84, 126, 126, 84, 36, 9, 1] olur ve 2^{63} sayısını gösterir. Merkez 126 < 127 iken bir sonraki satırın merkezi 252'dir. Böylece N(7) = 9 olur. Bu örnekte ilk notun p + 2 seçimi sınıra tam ulaşır.

Örnek 4. Güvenli satırın ötesine geçmek

p = 38 için durum farklıdır:

Satır	Merkez katsayısı	Taban 274877906943 ile karşılaştırma
40	137846528820	Küçük, haneler korunur
41	269128937220	Küçük, haneler korunur
42	538257874440	Büyük, elde gerekir

N(38) = 41. Dolayısıyla p + 2 = 40 güvenli bir seçimdir, ama son geçerli seçim değildir. Bir dönüştürücü bu tabanda 41. satırı kabul edebilir, 42. satırın katsayılarını doğrudan hane diye kabul edemez.

Örnek 5. İstenen satır için taban seçmek

42. satırı elde taşımadan yazmak istediğimizi düşünelim. Merkezi 538257874440’tır. p = 38 tabanı yetersizdir. p = 39 için taban 549755813887 olur ve merkez bu tabandan küçüktür. Bu nedenle Mersenne tabanları arasında yeterli en küçük p değeri 39’dur. N(39) = 42 olduğu da bir sonraki merkezin 1052049481860 olmasıyla doğrulanır.

Genel karar kuralı şudur: n satırı seçildiğinde c(n) < 2^p − 1 sağlayan en küçük p ≥ 2 aranır. Bu, sabit p için N(p) bulma sorusunun tersidir. Bütün katsayıları ayrı ayrı sınamak yerine en büyüğünü sınamak yeterlidir.

Örnek 6. Bir milyon bitten uzun bir sayı

p = 1000 için taban $b = 2^{1000} - 1$ ve son satır N(1000) = 1005’tir. Bu satırın başı ve sonu şöyledir:

[1, 1005, 504510, …, 504510, 1005, 1].

Bu gösterimde toplam 1006 hane vardır. Ortadaki … işareti sıfırları değil, burada yazılmayan 1000 Pascal katsayısını temsil eder. En büyük katsayı onluk yazımda 301 rakamlıdır. Bütün katsayılar tabandan küçüktür. Aynı hanelerin konum değerleriyle gösterdiği sayı $2^{1\,005\,000}$ olur.

Bu sayı ikilik tabanda bir adet 1 ve arkasında 1 005 000 adet 0 ile yazılır. Dolayısıyla tam ikilik uzunluğu 1 005 001 bittir. Buradaki yarar, büyük bir sayının hangi Pascal satırıyla eldesiz temsil edilebildiğini kesin olarak bilmektir. Onluk gösterimin bütün rakamlarını basmaya gerek yoktur.

1006 hane kullanmak kendiliğinden veri sıkıştırması değildir. Her hane 1000 bite kadar yer isteyebilir. Hanelerin sayısı ile onları saklamak için gereken bit sayısı farklıdır.

Örnek 7. Ham ikilik yazımı yaklaşık 125 GB olan sayı

p = 1 000 000 seçilsin. Eşikler 667533 ≤ p < 2670166 olduğu için N(p) − p = 10 ve N(p) = 1 000 010 olur. Taban $b = 2^{1\,000\,000} - 1$’dir.

[1, 1000010, 500009500045, …, 500009500045, 1000010, 1].

Satırda toplam 1 000 011 hane vardır. Gösterilmeyen orta bölüm 1 000 005 katsayı içerir. Temsil edilen sayı $2^{1\,000\,010\,000\,000}$ ve tam ikilik uzunluğu 1 000 010 000 001 bittir.

Bu bitleri sıkıştırmadan, sekiz bitlik baytlar halinde saklamak 125 001 250 001 bayt, yaklaşık 125 GB gerektirir. Buna karşılık “2 üzeri 1 000 010 000 000” ifadesi kısa bir sembolik tanımdır. Sembolik tanım ile sayının bütün bitlerini belleğe açmak aynı işlem değildir. Bu örnekte bütün büyük sayı veya satır üretilmiş değildir. Son satır, matematiksel eşik sonucundan belirlenmiştir.

Örnek 8. Bir trilyon parametresi

$p = 10^{12} = 1\ 000\ 000\ 000\ 000$ olsun. Bu ölçekte kısa bir tabloyu bütün satırları deneyerek üretmek yerine kanıtlanmış hata bandını kullanabiliriz. $h(p) = \tfrac{1}{2}\log_2(\pi p/2)$ için $20.257 < h(p) < 20.258$ olur. Matematik notundaki $-1 < E(p) \le 0.270575$ sınırıyla

$$19.257 < N(p) - p < 20.529.$$

$N(p) - p$ bir tamsayıdır. Bu aralıkta yalnız 20 vardır. Dolayısıyla $N(p) = 10^{12} + 20 = 1\ 000\ 000\ 000\ 020$.

$n = 10^{12} + 20$ yazarsak satırın düzeni $[1, n, n(n-1)/2, \ldots, n(n-1)/2, n, 1]$ olur. Toplam $10^{12} + 21$ hane vardır. Temsil edilen sayı $2^{10^{24} + 20 \times 10^{12}}$ olur. Bu sonuç bütün katsayıların tek tek hesaplandığı anlamına gelmez. Son satırın hangi satır olduğuna, kanıtlanmış sınırın izin verdiği tek tamsayı seçilerek karar verilmiştir.

Örnek 9. p’nin kendisi 101 rakamlı olduğunda

Şimdi $p = 10^{100}$ olsun. Bu, onluk yazımda bir adet 1 ve ardından 100 adet 0 demektir. Taban $b = 2^{10^{100}} - 1$ olur. $h(p)$ için $166.422 < h(p) < 166.423$ sınırları geçerlidir. Aynı hata bandı

$$165.422 < N(p) - p < 166.694$$

verir. Tek olası tamsayı 166’dır. Dolayısıyla $N(10^{100}) = 10^{100} + 166$ kesin olarak belirlenir.

$n = 10^{100} + 166$ olmak üzere bu dev satır yine

$$[1, n, n(n-1)/2, \ldots, n(n-1)/2, n, 1]$$

biçimindedir. Tam hane sayısı $10^{100} + 167$’dir. Temsil edilen sayı $2^{10^{200} + 166 \times 10^{100}}$ olur. Başındaki ve sonundaki 1, Pascal satırının uçlarıdır. Aradaki … bilinmeyen veya keyfî değerler değil, n satırının burada yazılmayan belirli binom katsayılarıdır.

Bu örnekte $N(p)$’nin yararı özellikle açıktır: yazdırılması mümkün olmayan bir satır ailesinin eldesiz geçerlilik sınırını matematikle belirler. Satırın tamamını saklayabildiğimiz veya hesaplayıp ekrana döktüğümüz iddia edilmez.

Örnek 10. Programda kullanılabilecek sınır çiftleri

p	N(p)	İlk eldesiz olmayan satır
2	2	3
3	4	5
4	5	6
7	9	10
8	10	11
37	39	40
38	41	42
39	42	43
100	103	104
158	161	162
159	163	164
1000	1005	1006

Tablonun her satırı iki kontrol sunar: N(p) satırı sığmalı, N(p) + 1 satırı sığmamalıdır. Bu çiftler taban dönüşümü veya Pascal katsayılarını hane olarak kullanan bir programın sınır denetimlerinde işe yarar. p değişince taban da değişir. Tablodaki sıçramalar aynı tabanda satır atlandığı anlamına gelmez.

Kullanımın kapsamı

N(p), bu sayı gösteriminin kesin çalışma aralığını belirleyen matematiksel bir araçtır. Eğitimde elde taşımayı açıklamak, algoritmada doğru tabanı seçmek ve sınır durumlarını denetlemek için kullanılabilir. Bütün bir Pascal satırını veya çok büyük kuvveti üretmenin daha hızlı olduğunu tek başına göstermez. Ayrı bir şifreleme, genel sıkıştırma veya asallık testi sonucu sunmaz. N(p)’nin hesaplanmasının maliyeti de kullanılan yönteme bağlıdır.

9. Adreslerden hücrelere: PCCPU ile işlem modeli

9.1. Mimari hedef

Buradaki hedef, sayıları bir matris veya çok boyutlu hücre düzeninde adreslemek ve bu adresler üzerinden aritmetik yürütmektir. Hücre yalnız bir sayı tutmak zorunda değildir. Bir formülü, bekleyen işlemi, sonucu veya bir çekirdeğin erişilebilir durumunu temsil edebilir. Bir hücreye yerel hesap birimi bağlanması veya o hücrenin bir alt hücre grubunu temsil etmesi de tasarım seçenekleridir.

Bu görüşte çekirdeklerin girdileri, sonuçları ve çalışma durumları ortak bir adres düzeninde görünür olur. PCCPU bu düzen üzerinde işi başlatır, gerekli adresleri seçer ve tamamlanma olaylarını izler. Sayı adresleme çalışması ile işlemci fikrinin bağlantısı budur. N(p) çalışması bu mimarinin tamamı değildir. Matematiksel gösterimin eldesiz sınırını sağlayan bileşenidir.

Mevcut PCCPU belgesi, işi dağıtan ve çekirdeklerin bağımsız çalışmasına izin veren bir denetleyici tasarımını anlatır [4]. Buradaki ortak hücre adresi ve küçük yürütme örneği, bu fikri açıklamak için önerilen bir modeldir. Hunter'a uygulanmış bir çok çekirdekli donanım veya tamamlanmış bir komut kümesi olarak sunulmaz.

9.2. Aynı adreste ne bulunur?

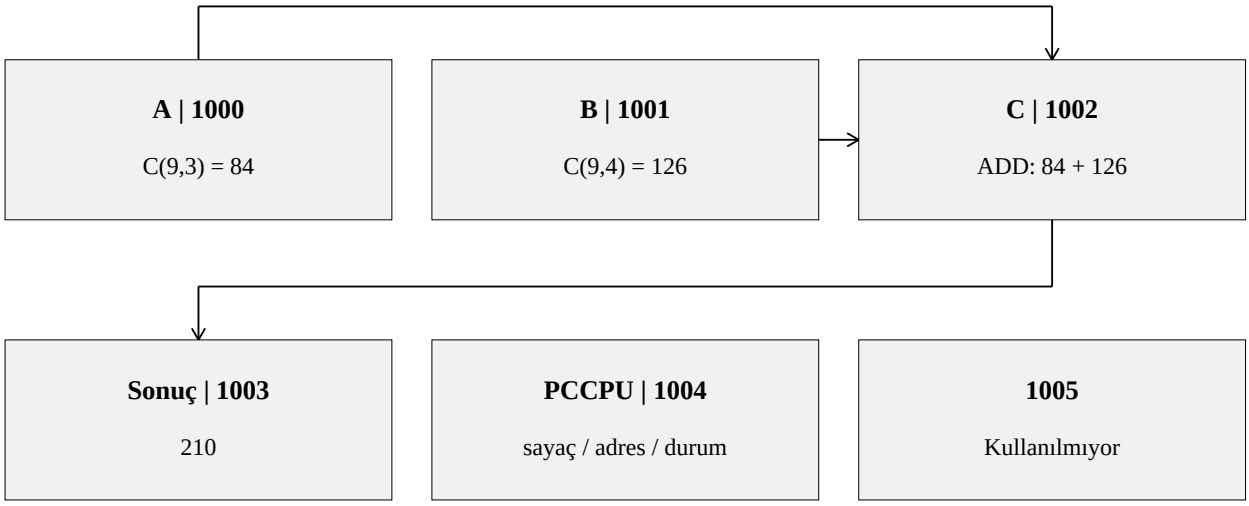
"Bir hücrede işlemci bulunabilir" düşüncesi iki düzeyde ele alınabilir. Fiziksel tasarımda hücreye bellek ve hesap birimi birlikte yerleştirilebilir. Mantıksal modelde ise hücrenin adresi, belirli bir çekirdeğin iş kaydını ve durumunu gösterir. Aşağıdaki örnek ikinci düzeyi uygular. Fiziksel devre yerleşimini belirlemez.

Hücre alanı	Görevi
Tür	Veri, çalışan birim, denetleyici veya kullanılmayan hücre
Matematik adresi	Örneğin C(9,3), yani 9. satırın 3. katsayısı
İşlem ve girişler	Katsayı üret, iki hazır değeri topla, sonucu hedefe yaz
Sahip	İşi yapan mantıksal çekirdek
Durum	Bekliyor, tamamlandı veya sonuç hazır
Değer	Üretilmiş sayı; henüz üretilmediyse boş

Bir RAM eşlemesi, bu kayıtlara adres üzerinden erişim sağlar. Bir kaydı adreslemek tek başına sıradan RAM'i hesap yapan bir devreye dönüştürmez. İşlemi yürüten birim ve sonucun yazılma kuralı ayrıca gerekir. Bununla birlikte veri ve yürütme durumunun aynı adres düzeninden görülebilmesi, tasarımın somut bir parçasıdır.

9.3. İki satırlı küçük hücre düzeni

Örnekte iki satır ve üç sütun vardır. Hücre kaydı adresi 1000 + 3r + c olarak seçilir. Buradaki r ve c matris koordinatlarıdır. Pascal satırı n ve katsayı konumu k ile aynı koordinatlar değildir. Adresler kayıt kimliğidir; bayt genişliği veya fiziksel bellek yerleşimi tanımlanmış değildir.



Şekil 3. Aynı hücre düzeninde üç mantıksal işçi, bir sonuç hücresi ve PCCPU durum kaydı. Oklar veri bağımlılığını gösterir.

1000 adresindeki A işçisi C(9,3) = 84 değerini üretir. 1001 adresindeki B işçisi C(9,4) = 126 değerini üretir. 1002 adresindeki C işçisi bu iki kaydı giriş olarak kullanır. İkisi de hazır olunca toplar ve 210 sonucunu 1003 adresine yazar. PCCPU durumu 1004 adresinden görülür. Son hücre kullanılmaz.

Burada “işlemciler arası aritmetik”, çekirdek kimliklerinin toplanması değildir. Ayrı işçilerin ürettiği değerlerin adresler aracılığıyla birbirine bağlanmasıdır. A ve B farklı sırada tamamlanabilir. C iki girdinin de hazır olmasını beklemek zorundadır.

9.4. Sayaç, adres ve tamamlanma

Bu küçük modelde denetleyici kaydı üç ayrı bilgi tutar: mantıksal değerlendirme turu, tamamlanan iş olayı sayısı ve son seçilen iş hücresinin adresi. Bunlar PCCPU’nun nihai register adları veya bit düzeni değildir. Sıra, adres ve durumun nasıl birlikte izlenebileceğini gösteren bir seçimdir.

Aşama	Hücrelerde olan iş	Denetleyicide görülen durum
Başlangıç	A, B ve C bekliyor. Sonuç boş.	Olay sayısı 0, seçili adres yok
İlk hazır küme	A, 84 üretir. B, 126 üretir. C henüz çalışmaz.	İki tamamlanma olayı kaydedilir
İkinci hazır küme	C, 84 + 126 = 210 hesaplar ve hedefe yazar.	Üçüncü olay kaydedilir
Bitiş	1003 adresindeki sonuç hazırdır.	İş tamamlandı, yeniden başlatılmaz

İki mantıksal tur, iki fiziksel saat çevrimi demek değildir. Örnek Python içinde sıralı çalışır. Hazır girdilerin aynı durum görüntüsünden seçilmesi, bağımlılığı görünür kılar. Gerçek donanımda hesap gecikmesi, haberleşme, bellek erişimi ve eşzamanlama ayrıca tasarlanmalıdır.

PCCPU’nun her çekirdeğin her aritmetik adımını merkezden yönetmesi gerekmez. Mevcut tasarımdaki gibi işi dağıtıp çekirdeklerin yerel sayaçlarla ilerlemesi mümkündür [4]. Bu modeldeki sayaç ve adres kaydı, o hedefi anlatmak içindir. Merkezde darboğaz olmayacağı kanıtlanmış değildir.

9.5. Aritmetik bir adres dönüşümüne ne zaman iner?

Örnekte yalnız sonuç 210 değildir. Aynı zamanda bir Pascal adresi ilişkisi vardır:

C(9,3) + C(9,4) = C(10,4) = 210.

Genel kural C(n,k) + C(n,k+1) = C(n+1,k+1) olduğundan, uygun iki komşu katsayının toplamı hedef katsayının adresiyle ifade edilebilir. Bir sistem, değeri hemen hesaplamak yerine bu ilişkiyi de saklayabilir. Küçük yürütme modeli ise doğrulamayı açık tutmak için sayıları gerçekten üretip toplar.

Her iki adresin toplamı tek bir Pascal adresine dönüşmez. İlişki yoksa bir toplama ifadesi veya normal sayısal sonuç tutulur. Fiziksel RAM adreslerini toplamak da bu matematiksel kuralı uygulamak değildir. Adresin hangi nesneyi ve hangi işlemi temsil ettiği bilinmelidir.

Aynı tabandaki bütün satır sayıları için başka bir kural vardır. $R(b,n) = (b + 1)^n$ tanımıyla $R(b,r) \times R(b,s) = R(b,r+s)$. Örneğin onluk tabanda satır 2 ile satır 2'nin çarpımı, satır 4'ün sayısını verir: $121 \times 121 = 14641$. Satır 3 ile satır 2'nin çarpımı ise $1331 \times 121 = 161051$ olur.

İkinci çarpımda sayısal sonuç doğrudur, fakat Pascal'ın 5. satırındaki [1, 5, 10, 10, 5, 1] katsayıları onluk hane değildir. Elde gerekir. Mersenne tabanı $b = 2^p - 1$ kullanıldığında, doğrudan katsayıları hane diye okumak için $r + s \le N(p)$ koşulu aranır. Onluk taban Mersenne tabanı değildir; orada genel merkez koşulu $C(r+s,\lfloor (r+s)/2 \rfloor) < 10$ kullanılır.

9.6. N(x,y,z)[a] gösteriminin yeri

N(x,y,z)[a], satır, giriş ve konum ilişkisini yön seçimiyle birlikte adresleme önerisidir. [a] için 0 soldan, 1 sağdan seçim düşünülür. z = 0 merkez seçimi olabilir. Konuşulan örneklerde x hane sayısını gösterdiği için Pascal satır numarası x − 1 olur. y alanına girilen 1 ise tabanla özdeşleştirilmez. Alanların tam anlamı ve veri türleri henüz sabitlenmiş değildir.

Hane okuma, konuma göre kuvvet alma ve basamak katkısı ayrı işlemlerdir. 14641 içindeki iki 4'ü okumak iki durumda da 4 verir. Soldan sıfırla başlayan konumu üs yapmak seçilirse sonuçlar 4^1 ve 4^3 olur. Onluk basamak katkıları ise soldaki 4 için 4×10^3, sağdaki 4 için 4×10^1 olur. Yalnız yön bitini değiştirmek bu üç işlemden hangisinin istendiğini belirlemez. İşlem türü açık olmalıdır.

Bu nedenle küçük yazılım modeli belirsiz N gösterimini bir komut standardı gibi uygulamaz. Matematik adresi olarak C(n,k), yürütme hücresi için matris koordinatı, kayıt erişimi için adres numarası kullanır. İleride bunların ortak bir adres sözcüğüne nasıl kodlanacağı tasarım kararıdır.

9.7. Somut örneği çalıştırmak

Depo kökünden aşağıdaki komut küçük modeli çalıştırır:

```
python3 examples/pccpu_address_model.py
```

Örnek A, B ve C işçilerinin ürettiği değerleri, hedef adresi ve denetleyici durumunu gösterir. Sonuç 210’dur. Sonuç hazır olmadan okunamaz ve tamamlanan iş ikinci kez yürütülmez. Satır çarpımı örnekleri de sonucun eldesiz olup olmadığını ayrı bildirir.

Bu bir yazılım açıklama modelidir. Gerçek çekirdekler arası veri yolu, cache tutarlılığı, kesme mekanizması, bellek koruması veya saat devresi içermez. Fiziksel uygulama için hücrenin veri genişliği, hazır olma protokolü, birden fazla yazıcının düzeni ve sayaç ilerletme koşulları belirlenmelidir.

Matematiksel temel, adresler arasındaki geçerli dönüşümlerdir. Mimari öneri, değer ve iş durumunu hücre düzeninde birleştirmektir. N(p)’nin somut rolü ise seçilen gösterimin eldesiz sınırını denetlemektir. Bu üç parça birlikte bir işlem modeli oluşturabilir; hız ve enerji avantajı ayrıca gösterilmelidir.

10. Yeniden üretim ve sonuç

Ana Python ve C programları n = p + 2 satırını üretir. Yeni açıklama bunların davranışını değiştirmez. N(p) kontrolleri ayrı doğrulayıcı üzerinden yürütülür. Depo kökünden aşağıdaki komutlar kullanılabilir:

```
python3 mersenne_rows.py 7 full
python3 -B tests/verify_np.py --quick
python3 -B -m unittest discover -s tests -p 'test_*.py'
```

Tam matematik kontrolünün komutu `python3 -B tests/verify_np.py` biçimindedir. Doğrudan tarama $p \le 20000$, eşik denetimi $k \le 12$ kapsamındadır. Sonlu bir hesap kontrolü, bütün olası p değerleri için kanıt yerine geçmez.

İlk çalışma, Pascal satırlarından belirli bir eldesiz sayı gösterimi kurar. N(p) eki, bu gösterimin sabit tabandaki son geçerli satırını belirler. İkinci çalışma ilkini değiştirmez, onun sınırını inceler. Açıklamanın temel ayrımı, geçerli bir örneğin bulunması ile son geçerli örneğin bulunması arasındadır.

Kaynakça ve kullanım koşulları

[1] James Augustus. *Mersenne tabanlarında Pascal satırları*. Özgün not: Türkçe metin ve Türkçe PDF. Depo: pascal-mersenne-rows.

[2] James Augustus. *Mersenne tabanlarında en büyük eldesiz Pascal satırı*. N(p) eki: Türkçe metin ve Türkçe PDF.

[3] *N(p) hesap denetimi*. Doğrulayıcı. Doğrudan tarama $p \le 20000$, eşik denetimi $k \le 12$.

[4] James Augustus. PCCPU tasarım notu. İş dağıtma, çekirdeklerin yerel sayaçları ve açık mimari kararlar.

Bu açıklama, deponun belge lisansı olan CC BY 4.0 kapsamında sunulur. Telif hakkı © 2026 James Augustus. Tez düzeni, metnin sunum biçimini ifade eder. Bu belge bir üniversiteye sunulmuş veya onaylanmış lisansüstü tez olarak tanımlanmamaktadır.